ПРАКТИЧНА РОБОТА №3

Побудова характеризаційних тестів для legacy-модуля

Дисципліна: Комплексне перепроєктування та удосконалення програмного забезпечення

Варіант №6 - Express.js, JavaScript

Обраний модуль: lib/utils.js (272 фізичні рядки)

Студент: Шаровський Владислав Сергійович

Група: ІПЗ-21-1 ict

Репозиторій Express.js: https://github.com/expressjs/express

Код практичної №1: https://github.com/GitVlad14/Express-JS-Practical-1

Тести практичної №3: додаються до окремої гілки або репозиторію після публікації

Дата проведення локального тестування: 09.11.2026

2026

1. Мета та вибір модуля

Мета роботи - зафіксувати фактичну поведінку наявного JavaScript-модуля за допомогою характеризаційних тестів, дослідити типові та межові сценарії, виміряти покриття і підготувати підґрунтя для майбутнього безпечного рефакторингу.

1.1. Чому обрано utils.js

Модуль lib/utils.js містить 272 фізичні рядки, 14 функцій і 8 прямих залежностей за метриками практичної №1. Він відповідає рекомендованому обсягу 200-600 рядків, використовується іншими частинами Express.js і має нетривіальні розгалуження. Максимальна складність функції становить M = 7. Показник невисокий, але модуль зручний для ізольованого охоплення сценаріїв.

Функції, що аналізуються: normalizeType, normalizeTypes, compileETag, compileQueryParser, compileTrust, setCharset; опосередковано перевіряються acceptParams, createETagGenerator, parseExtendedQueryString. Для модулів Express.js загалом існують тести. Тому тут створено окремий додатковий characterization-набір, а не стверджується, ніби в проєкті взагалі немає тестів.

1.2. Зв’язок із попередніми роботами

Практична №1 показала, що utils.js підключається з application.js і response.js та є спільною точкою допоміжної логіки. Практична №2 розглядала ризики залежностей у цих частинах. Створення тестів для utils.js зменшує ризик випадково змінити обробку параметрів, ETag, типів вмісту та довірених проксі.

2. Вивчення фактичної поведінки

Перед тестуванням використано вихідний текст utils.js, збережений у JSON-звіті ESLint під час практичної №1. Копія із заголовком MIT License включена в source_snapshot/utils.js. Для кожного експортованого методу перевірено аргументи, альтернативні гілки та делегування до зовнішніх бібліотек.

Тестовий фреймворк - стандартний node:test, інструмент покриття - вбудоване V8 coverage (Node.js 22). У методичних рекомендаціях наведено Jest + Istanbul, проте обраний засіб також вимірює покриття рядків і не потребує завантаження npm-пакетів. Вибір зроблено для відтворюваності у Windows 10.

Зовнішні залежності content-type, etag, mime-types, proxy-addr і qs замінені контрольованими тестовими двійниками. Це дає змогу тестувати розгалуження власне utils.js незалежно від мережі та версій npm. Повноцінну інтеграцію з реальними пакетами ці тести не перевіряють.

2.1. Метод Фезерса

Для кожного входу зафіксовано фактично спостережене значення або помилку, після чого створено тестове твердження. Мета тесту - зберегти поточну поведінку, навіть якщо вона здається суперечливою. Виправляти виробничий код у цій практичній не потрібно.

2.2. Межові та помилкові випадки

Набір охоплює нормальні аргументи, порожні рядки й масиви, null, недійсні типи MIME, неправильні режими налаштувань, відсутні параметри, повторні ключі та порогові значення для числа довірених проксі. Кожен тест ізольований і не змінює код Express.js.

3. Набір характеризаційних тестів

Виконано 34 автоматичні тести. Таблиця нижче стисло описує перевірені групи. Повний програмний код із окремими assert-твердженнями наведено у файлі tests/characterization.test.cjs.

Тести	Ділянка	Вхідні дані	Спостережений результат	Категорія
1-4	HTTP methods, MIME тип	GET/POST; html/json; невідомий тип	Нижній регістр; розпізнавання MIME; резервний octet-stream	Типові + межові
5-10	Параметри типу	q=0.4; q=; повтор q; malformed; []	q числовий; для порожнього q отримано NaN; останній q переважає	Межові + підозрілі
11	normalizeType	null	Виняток TypeError	Некоректні дані
12-18	compileETag і ETag	callback, true, false, weak, strong, Buffer, hex	Вибір функції, disable, помилка, ETag від буфера	Типові + помилки
19-23	compileQueryParser	custom, simple, false, extended, invalid	Вибір обробника, передавання allowPrototypes=true, виняток	Типові + підозрілі
24-29	compileTrust	function, true, 2 hops, CSV IP, false, array	Функція, довірені хопи, trim, делегування	Межові + типові
30-34	setCharset	порожній тип/charset, наявний charset, invalid	Повернення без змін, заміна charset, виняток	Межові + помилки

Тести не просто запускають функцію: кожен має assert, що порівнює фактичний результат із збереженою поведінкою або перевіряє виняток. Повний перелік назв наведено в results/test_output.txt.

3.1. Приклади конкретних тверджень

normalizeType('html') -> {value:'text/html', params:{}}; compileETag('invalid') -> TypeError; compileTrust(2) повертає true при hop index 0 та false при index 2; setCharset('text/html', 'utf-8') -> 'text/html; charset=utf-8'.

4. Вимірювання тестового покриття

Початкове вимірювання виконано окремим baseline-тестом, який лише завантажує модуль, не перевіряючи його методи. Через ініціалізацію експорту та визначення функцій стартове рядкове покриття за V8 становило 54,61%, а не 0%. Це результат інструмента, а не припущення.

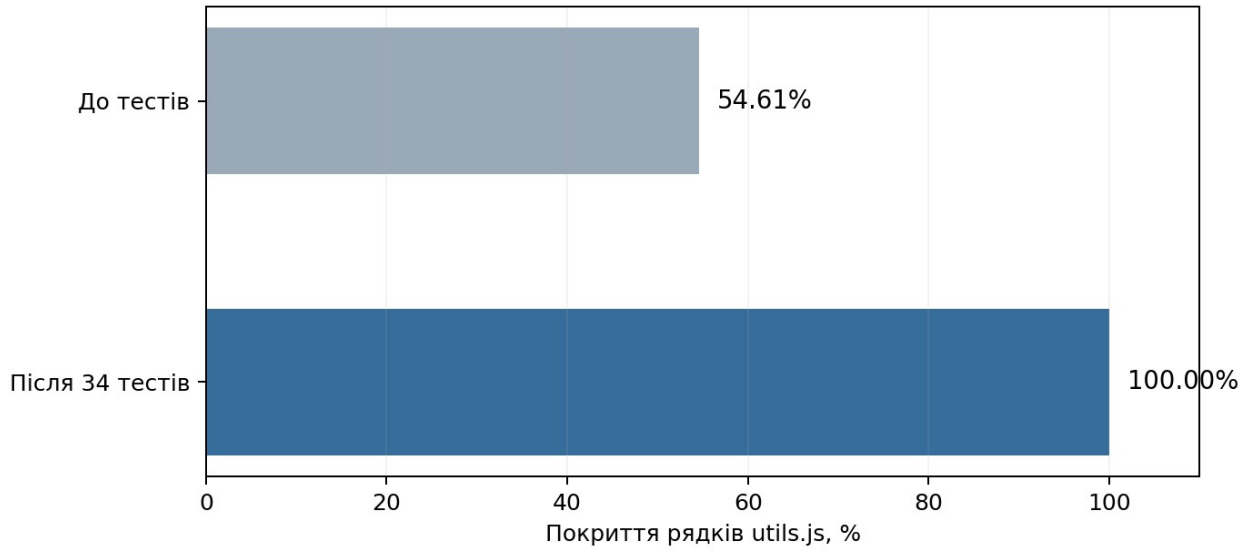


Рисунок 1 - покриття рядків utils.js на збереженому знімку вихідного коду.

Показник	До створення тестів	Після 34 тестів
Рядкове покриття utils.js	54,61%	100,00%
Покриття функцій utils.js	18,18%	100,00%
Тестів у запуску	1 baseline-тест	34 характеризаційні тести
Успішно / помилки	1 / 0	34 / 0

Вимога не менше 70% рядків досягнута для перевіреного знімка utils.js. Значення є результатом тестового запуску з підміною зовнішніх бібліотек; зафіксоване покриття гілок 100% відображає звіт V8 у цій конфігурації й не означає повного охоплення усіх інтеграційних умов.

Докази: results/baseline_output.txt і results/test_output.txt з таблицями coverage, а також source_sha256.txt, який ідентифікує перевірений текст.

5. Підозрілі результати та обмеження

5.1. Порожній коефіцієнт якості q

Для normalizeType('text/html;q=') якість виявилася NaN, тому що parseFloat('') повертає NaN. Поведінку зафіксовано окремим тестом, але не оцінено як правильну для всіх клієнтів. Потенційний наслідок - некоректне сортування пропозицій типів вмісту у коді, що використовує цю метрику.

5.2. Повторний параметр q

Для normalizeType('text/html;q=0.2;q=0.9') результат має quality = 0.9: значення перезаписується останнім входженням. Це може залежати від домовленостей щодо розбору HTTP-заголовків. Поведінку збережено, не змінюючи реалізацію.

5.3. Режим extended і allowPrototypes

compileQueryParser('extended') передає до qs.parse параметр allowPrototypes: true. Це реальна властивість початкового коду, яку перевіряє тестовий двійник. Потенційну небезпеку роботи з особливими ключами об'єктів варто додатково оцінити інтеграційно; цей тест сам по собі не доводить уразливості.

5.4. Межі достовірності

Покриття вимірювалося на збереженій копії utils.js із результатів попередньої роботи. Коміт Express.js, раніше наданий для GitHub, не був криптографічно звірений із цим знімком. У комплект включено перемикач EXPRESS_UTILS_PATH, щоб на комп’ютері студента повторити перевірку безпосередньо на локальному клоні і звірити SHA-256.

6. Відтворення результатів

Після розпакування відкрити папку Practical_3_Express_Sharovskyi у Visual Studio Code та виконати npm.cmd test, npm.cmd run baseline, npm.cmd run coverage. Для цього необхідний Node.js 22 або новіший; зовнішні бібліотеки встановлювати не потрібно.

Для перевірки безпосередньо локального клону Express.js у PowerShell виконати:

$env:EXPRESS_UTILS_PATH="C:\Users\vlads\VSProjects\express\lib\utils.js"

npm.cmd run coverage

Remove-Item Env:EXPRESS_UTILS_PATH

Після успішного виконання GitHub-репозиторій має містити тести, package.json, знімок джерела, результати покриття, а PDF надсилається на навчальну платформу. Для практичної №3 викладач вимагає посилання на репозиторій із тестами та конфігурацією покриття.

7. Висновки

У межах практичної роботи підготовлено 34 характеризаційні тести для модуля utils.js проєкту Express.js. У локальному запуску всі тести пройшли; рядкове покриття збереженого модуля підвищилося з 54,61% до 100,00%. Зафіксовано типові сценарії, межові значення, варіанти з помилками та три поведінки, які потребують окремої оцінки.

Після такого набору тестів локальну логіку модуля можна змінювати обережніше, контролюючи незаплановані зміни поведінки. Однак результати не заміняють інтеграційні тести з реальними npm-залежностями і не гарантують відсутності дефектів. Далі доцільно звірити знімок із поточним комітом та додати інтеграційні сценарії.

8. Джерела

1. Методичні рекомендації та завдання практичної роботи №3, 2026.

2. Express.js. https://github.com/expressjs/express

3. M. Feathers. Working Effectively with Legacy Code. 2004.

4. Node.js: Test runner, code coverage. https://nodejs.org/api/test.html

5. Результати практичної №1, metrics.json та ESLint JSON (09.11.2026).

Додаток А. Повний перелік тестів

Перелік відповідає файлу tests/characterization.test.cjs. Вхідні дані й фактичні результати наведено скорочено. Детальні assert-твердження див. у вихідному коді тестів.

№	Назва тесту	Вхідні дані	Зафіксована поведінка
1	HTTP methods are normalized to lowercase	METHODS	Нижній регістр
2	normalizeType maps common extension through MIME lookup	html	text/html
3	normalizeType maps unknown extension to octet-stream	weirdformat	application/octet-stream
4	normalizeType preserves explicit content type	application/json	Без змін
5	normalizeType parses q and a custom parameter	text/html; q=0.4; level=1	q=0.4; level=1
6	normalizeType ignores malformed segment missing equals	text/plain; broken	Некоректну частину пропущено
7	normalizeType skips semicolon before equals in following segment	text/html; flag; q=0.3	q=0.3
8	normalizeType accepts empty q value as NaN (suspicious observation)	text/html;q=	NaN - підозріле
9	normalizeType uses last q when repeated (suspicious observation)	text/html;q=0.2;q=0.9	q=0.9 - останній параметр
10	normalizeTypes maps an array including empty array	[], [html,json]	Масив перетворений
11	normalizeType fails for null input	null	TypeError

Додаток А. Продовження таблиці тестів

№	Назва тесту	Вхідні дані	Зафіксована поведінка
12	compileETag returns an existing custom function unchanged	callback	Та сама функція
13	compileETag true and weak select weak generator	true, weak	Слабкий генератор
14	compileETag strong selects strong generator	strong	Сильний генератор
15	compileETag false disables generator	false	undefined
16	compileETag rejects unknown values	invalid	TypeError
17	strong and weak ETag differ and accept Buffer	abc, Buffer	Сильний / слабкий ETag
18	ETag generator accepts explicit encoding	6162, hex	ETag для 2 байтів
19	compileQueryParser accepts user supplied function	callback	Та сама функція
20	compileQueryParser simple and true use native parser	true, simple	Querystring обробник
21	compileQueryParser false disables query parsing	false	undefined
22	compileQueryParser extended delegates with allowPrototypes=true (review)	extended, a=1	allowPrototypes=true

Додаток А. Продовження таблиці тестів

№	Назва тесту	Вхідні дані	Зафіксована поведінка
23	compileQueryParser invalid option throws	invalid	TypeError
24	compileTrust returns user supplied function unchanged	callback	Та сама функція
25	compileTrust true trusts all addresses	true	Довіряє всі адреси
26	compileTrust number trusts hops below threshold	2 hops	hop 0=true, 2=false
27	compileTrust comma separated string trims entries	127.0.0.1, 10.0.0.1	Пробіли видалено
28	compileTrust false delegates empty list	false	Порожній список
29	compileTrust array forwards original entries	[127.0.0.1]	Масив передано
30	setCharset returns original if content type missing	empty type	Початкове значення
31	setCharset returns original if charset missing	empty charset	Початкове значення
32	setCharset inserts charset	text/html, utf-8	charset=utf-8
33	setCharset overwrites existing charset	charset=ascii	charset замінено
34	setCharset propagates invalid content type error	not-a-type	TypeError